

Unit-IV

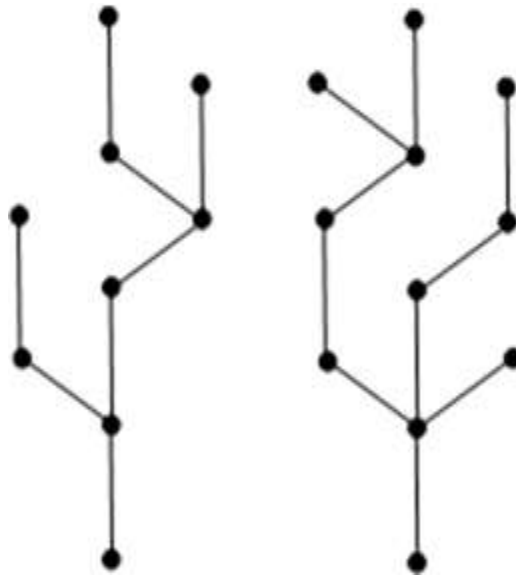
Unit-IV→ Trees: Trees mathematical properties, Binary Search Trees and their representation, expression evaluation, Complete Binary trees, Extended binary trees, Traversing binary trees, Searching, Insertion and Deletion in binary search trees, Complexity of searching algorithm, Path length, Huffman's algorithm, General trees, AVL trees, Threaded trees, B trees, Trie data structure.

Tree

A tree is a data structure consisting of nodes and edges, where an edge is incident upon exactly two nodes and there are no loops.

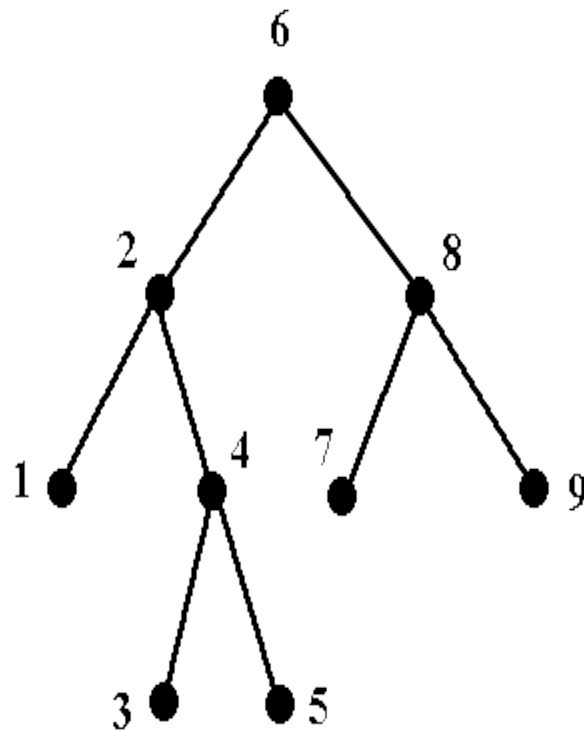
n nodes and $n-1$ edges

Nodes: leaf node, internal node



Rooted tree

A rooted tree is a tree in which parent-child relationship between nodes are defined and there exists exactly one node, called the root, which is the ancestor of all other nodes.



Root node has no parents.

An edge refers to the link from parent to child.

A node with no children is called leaf node.

Children of same parent are called siblings.

The set of all nodes at a given depth is called the level of the tree.

Root node is at level zero.

Depth of the root node is zero.

If there is a path from a node p to node q and $\text{level}(p) < \text{level}(q)$, then p is an ancestor of q and q is a descendant of p .

Height of a node is the distance from that node to its farthest descendant node.

Height of a tree is the height of root node.

Sub-tree is a subset of a tree.

The internal path length is the sum, taken over all nodes of the tree, of the depth of each node.

The external path length is the sum, taken over all leaves of the tree, of the depth of each leaf.

Binary tree: Each node has up to 2 child nodes.

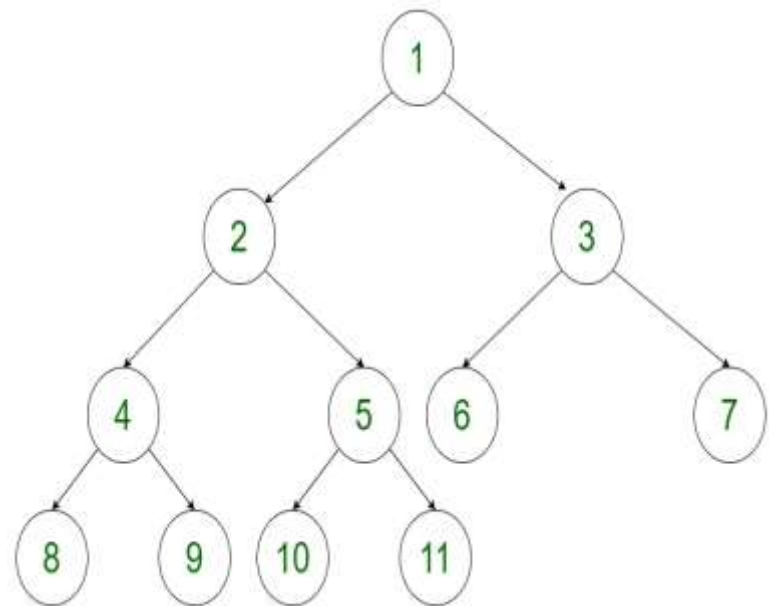
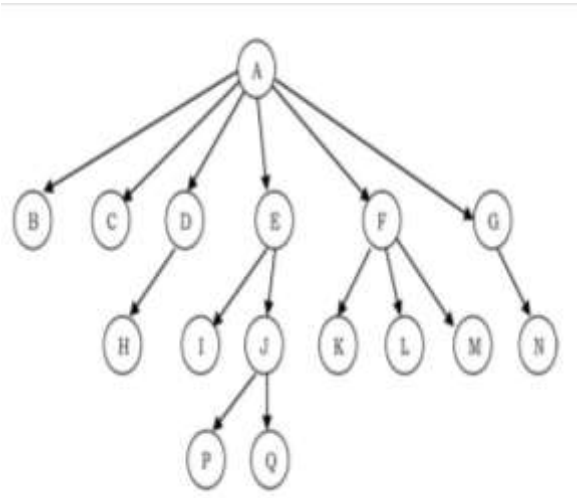
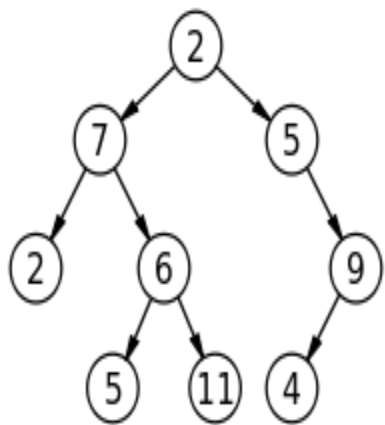
N-ary tree: Each node has up to N child nodes.

Complete binary tree: Each level is completely filled except possibly the last level where some nodes may be missing at the right end.

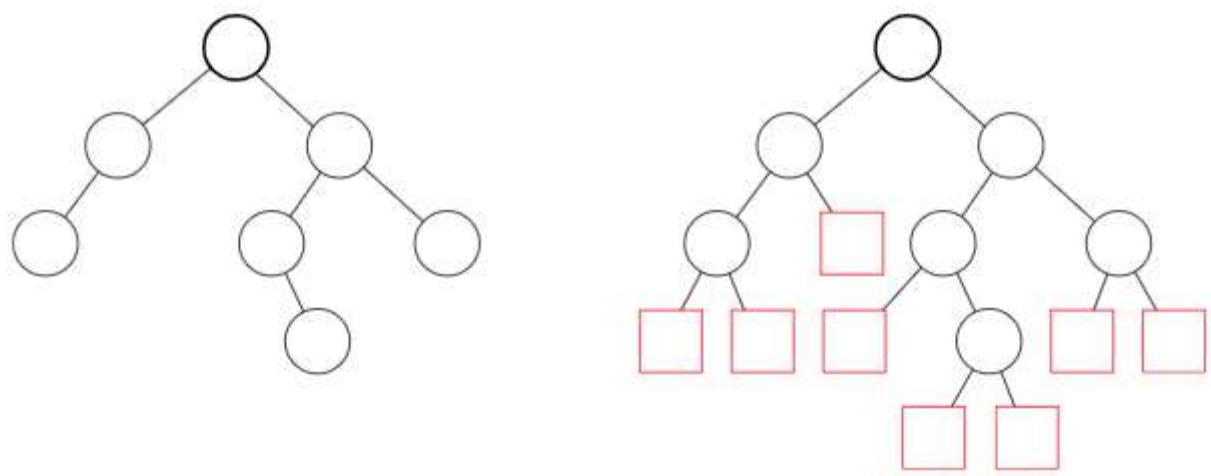
No. of nodes: 2^h to $2^{h+1}-1$, where h is the height of the tree.

Almost complete binary tree

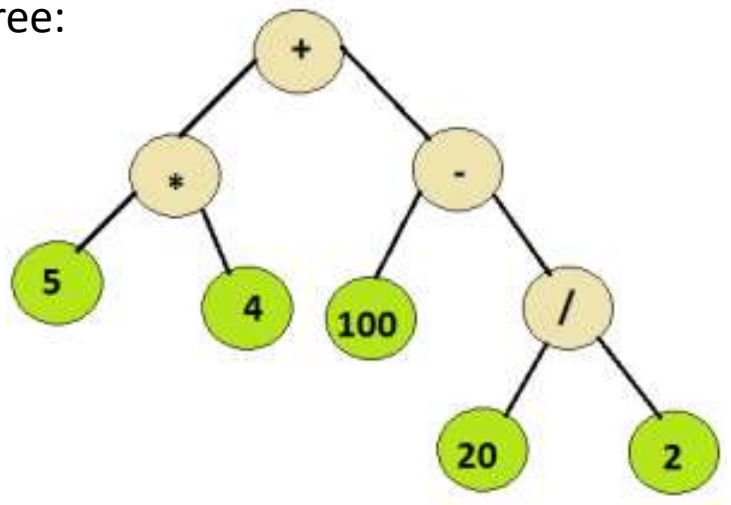
Heap is an almost complete binary tree in which every node has a bigger value than that of its children nodes.



Extended binary tree is a type of binary tree in which all the null sub tree of the original tree are replaced with special nodes called external nodes whereas other nodes are called internal nodes.



Expression evaluation using tree:



```
struct node // Structure declaration  
{int data;  
node *left, *right;  
};
```

```
node *root; // Declaration  
root=NULL; // Initialization
```

Task: To add a number x in a binary tree

Logic:

If root is NULL

Then make x the root

Else

Set current = root

Repeat until a node is created

Case 1: Both children are NULL

If x is odd then make it left child else make it right child

Case 2: Only one child is NULL

Then make x that child

Case 3: Both children exist

If x is odd then set current = left child else set current = right child

Tree traversal

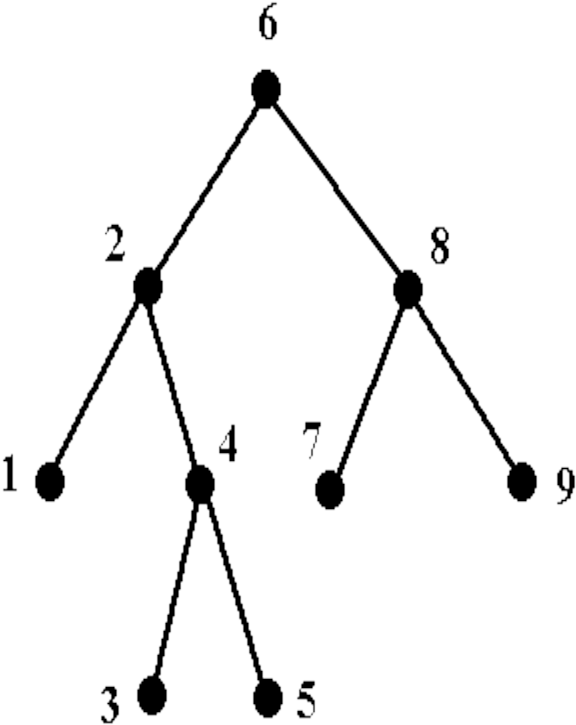
Pre-order: Current node, Left child node, Right child node

In-order: Left child node, Current node, Right child node

Post-order: Left child node, Right child node, Current node

Level-order

Example: Traverse the given tree.



Pre-order traversal: 6, 2, 1, 4, 3, 5, 8, 7, 9.

In-order traversal: 1, 2, 3, 4, 5, 6, 7, 8, 9.

Post-order traversal: 1, 3, 5, 4, 2, 7, 9, 8, 6.

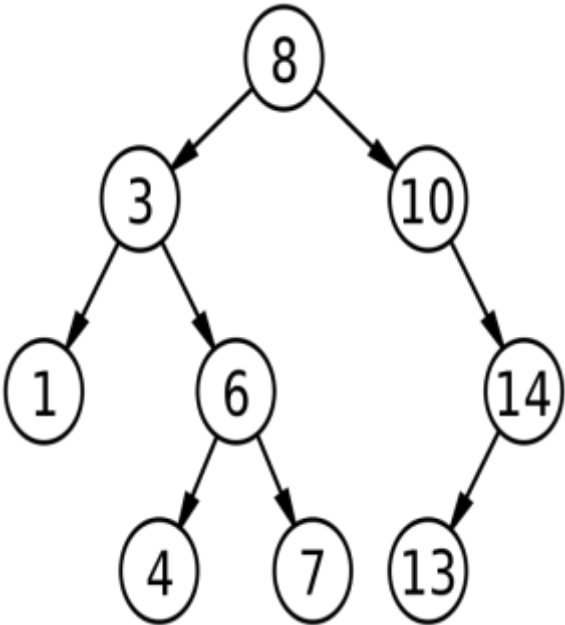
Level-order traversal: 6, 2, 8, 1, 4, 7, 9, 3, 5.

Example:

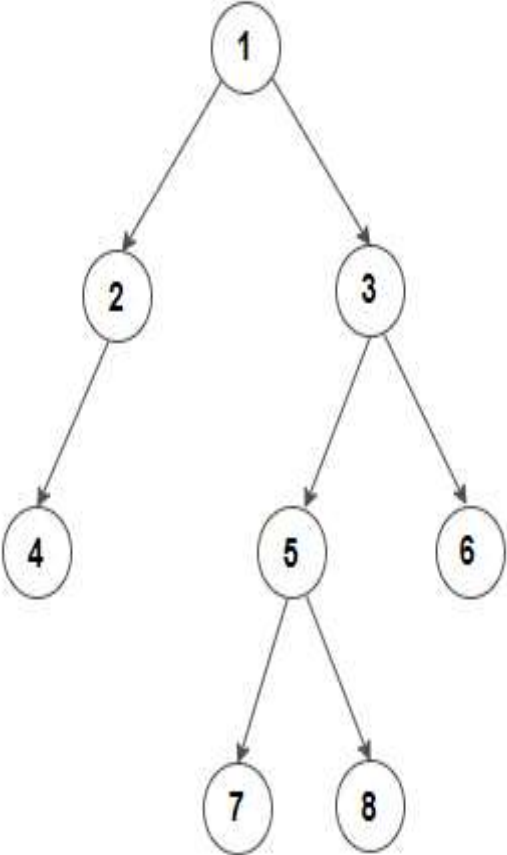
Pre-order traversal: 8, 3, 1, 6, 4, 7, 10, 14, 13.

In-order traversal: 1, 3, 4, 6, 7, 8, 10, 13, 14.

Construct the tree.



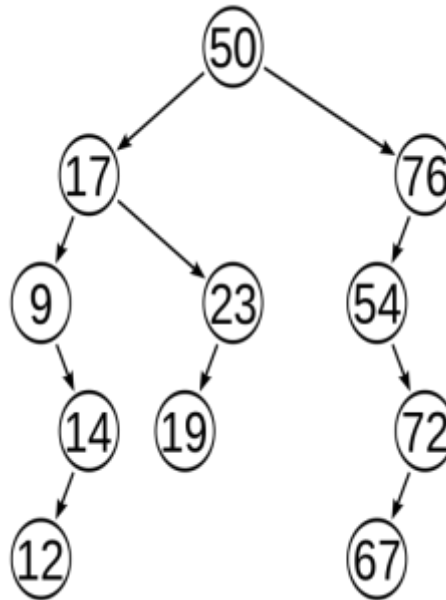
Example:
In-order traversal: 4, 2, 1, 7, 5, 8, 3, 6.
Post-order traversal: 4, 2, 7, 8, 5, 6, 3, 1.
Construct the tree.



Binary search tree

In a binary search tree, all nodes in the left sub-tree have lesser value than that of the current node and all nodes in the right sub-tree have greater value than that of the current node for all nodes.

Time taken to search a node = $O(\log n)$.



Huffman Coding

Given:

Alphabet, $A = \{a_1, a_2, \dots, a_n\}$

Weights, $W = \{w_1, w_2, \dots, w_n\}$

Weights are inversely proportional to probability

Calculate:

Codes, $C = \{c_1, c_2, \dots, c_n\}$

Minimize: $\sum_{i=1}^n w_i \times \text{length}(c_i)$

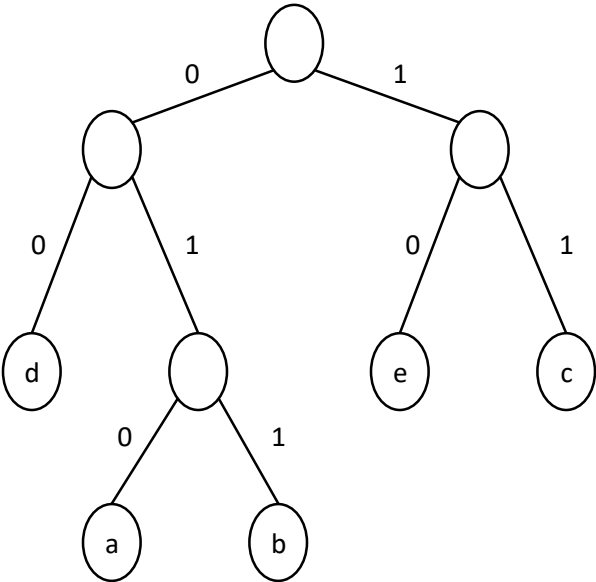
Note: c_i should not be a prefix of c_j for any i and j .

The simplest construction algorithm uses a priority queue where the node with lowest probability is given highest priority.

1. Create a leaf node for each symbol and add it to the priority queue.
2. While there is more than one node in the queue:
 - 2.1. Remove the two nodes of highest priority (lowest probability) from the queue
 - 2.2. Create a new internal node with these two nodes as children and with probability equal to the sum of the two nodes' probabilities.
 - 2.3. Add the new node to the queue.
3. The remaining node is the root node and the tree is complete.

Example:

a_i	a	b	c	d	e
w_i	0.10	0.15	0.30	0.16	0.29
c_i	010	011	11	00	10

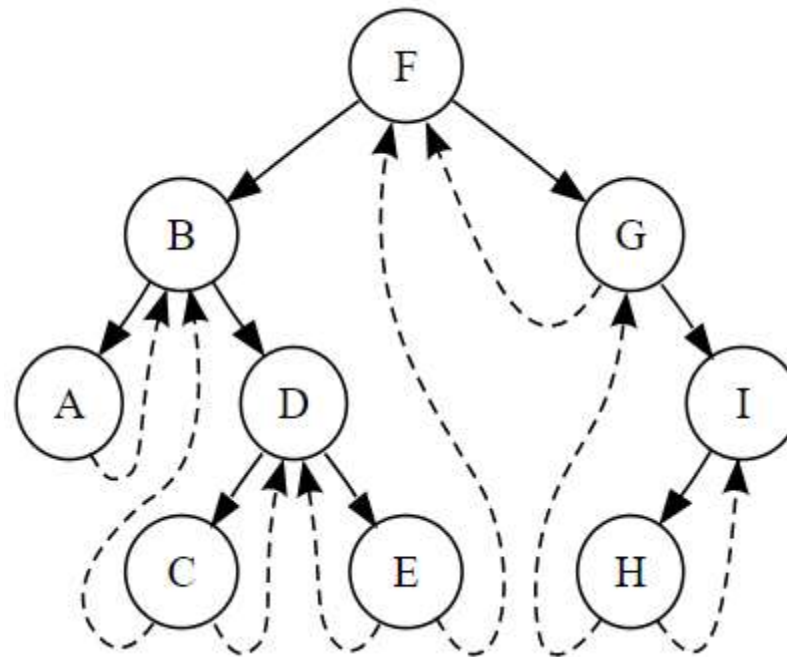


Encode: bedace → 011 10 00 010 11 10

Decode: 01110000101110 →

Threaded Tree

A binary tree is threaded by making all right child pointers that would normally be null point to the in-order successor of the node (if it exists), and all left child pointers that would normally be null point to the in-order predecessor of the node.

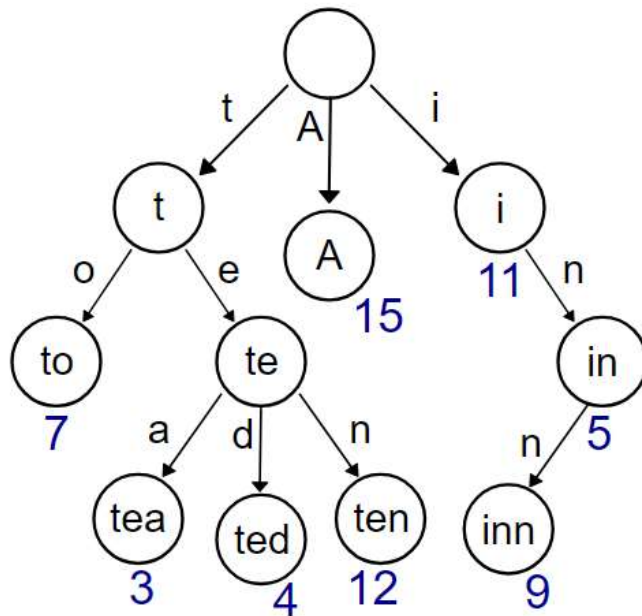


Trie

A trie, also called digital tree or prefix tree, is a type of k-ary search tree used for locating specific keys from within a set.

These keys are most often strings, with links between nodes defined not by the entire key, but by individual characters.

In order to access a key (to recover its value, change it, or remove it), the trie is traversed depth-first, following the links between nodes, which represent each character in the key.



Time complexity in big O notation

Operation	Average	Worst case
Search	$O(n)$	$O(n)$
Insert	$O(n)$	$O(n)$
Delete	$O(n)$	$O(n)$

Self-balancing Binary Search Trees

- AVL Tree
- B Tree

End of Unit